

scripts_sql

Scripts sql do projeto

- Scripts sql do projeto
 - Script SQL PostgreSQL para criar banco de dados e tabelas
 - Script SQL PostgreSQL para criar os relacionamentos
 - Script SQL PostgreSQL para criar os índices das tabelas
 - Compartilhar o acesso ao banco de dados
 - Ferramenta que pode ser usada para executar os scripts acima?
 - Referências

Script SQL PostgreSQL para criar banco de dados e tabelas

```
-- Cria banco de dados acm (Assistente virtual para médicos)

CREATE DATABASE avm;

-- operadores definition

CREATE TABLE IF NOT EXISTS operadores (
    Id serial primary key,
    nome varchar(50) NOT NULL,
    login varchar(50) NOT NULL,
    password varchar(50) NOT NULL,
    telefone varchar(20),
    CONSTRAINT operadores_login_key UNIQUE (login)
    --CONSTRAINT operadores_pkey PRIMARY KEY (id)
);

-- medicos definition

CREATE TABLE IF NOT EXISTS medicos (
    Id serial primary key,
    id_operadores integer NULL,
    nome varchar(50) NULL,
    telefone varchar(25) NULL,
    telefone_da_secretaria varchar(25) NULL,
    login varchar(50) NULL,
    senha varchar(20) NULL
    --CONSTRAINT medicos_pkey PRIMARY KEY (id)
);

-- medicos foreign keys

ALTER TABLE medicos ADD CONSTRAINT medicos_id_operadores_fkey FOREIGN KEY
(id_operadores) REFERENCES operadores(id) ON DELETE RESTRICT;
```

```
-- convenios definition
```

```
CREATE TABLE IF NOT EXISTS convenios (  
  Id serial primary key,  
  id_medico integer NULL,  
  nome varchar(50) not NULL,  
  login varchar(50) not NULL,  
  senha varchar(50) not NULL  
  --CONSTRAINT convenios_pkey PRIMARY KEY (id)  
);
```

```
-- convenios foreign keys
```

```
ALTER TABLE convenios ADD CONSTRAINT convenios_id_medico_fkey FOREIGN KEY  
(id_medico) REFERENCES medicos(id) ON DELETE RESTRICT;
```

```
-- clientes definition
```

```
CREATE table IF NOT EXISTS clientes (  
  Id serial primary key,  
  data_inclusão date not NULL,  
  id_convenio integer,  
  matricula_no_convenio varchar(50),  
  nome varchar(50) NOT NULL,  
  telefone_whatsapp varchar(25),  
  e_mail varchar(50),  
  login varchar(50),  
  senha varchar(20),  
  CONSTRAINT clientes_login_key UNIQUE (login)  
  --CONSTRAINT clientes_pkey PRIMARY KEY (id)  
);
```

```
-- clientes foreign keys
```

```
ALTER TABLE clientes IF NOT EXISTS ADD CONSTRAINT  
clientes_id_convenio_fkey FOREIGN KEY (id_convenio) REFERENCES  
convenios(id) ON DELETE RESTRICT;
```

```
-- formas_de_pagamento definition
```

```
CREATE TABLE IF NOT EXISTS formas_de_pagamento (  
  Id serial primary key,  
  nome varchar(30) NOT NULL  
  --CONSTRAINT formas_de_pagamento_pkey PRIMARY KEY (id)  
);
```

```
-- status_da_agenda_ou_consulta definition
```

```
CREATE TABLE IF NOT EXISTS status_da_agenda_ou_consulta (  
  Id serial primary key,  
  nome varchar(20) not NULL
```

```
name varchar(20) NOT NULL
--CONSTRAINT status_da_agenda_ou_consulta_pkey PRIMARY KEY (id)
);

-- expediente_do_medico_data definition

CREATE TABLE IF NOT EXISTS expediente_do_medico_data (
  id_medico integer NOT NULL,
  data_expediente date NOT NULL,
  hora_inicial time NOT NULL,
  hora_final time NOT NULL,
  FOREIGN KEY (id_medico) REFERENCES medicos(id) ON DELETE RESTRICT,
  PRIMARY KEY (id_medico, data_expediente, hora_inicial) -- chave
primária composta
);

-- expediente_do_medico_data foreign keys

ALTER TABLE expediente_do_medico_data ADD CONSTRAINT
expediente_do_medico_data_id_medico_fkey FOREIGN KEY (id_medico)
REFERENCES medicos(id) ON DELETE RESTRICT;

-- agenda definition

CREATE TABLE IF NOT EXISTS agenda (
  Id serial primary key,
  datetime_inclusao timestamp NOT NULL,
  id_medico integer NOT NULL,
  id_cliente integer NOT NULL,
  id_convenio integer,
  id_status_da_agenda_ou_consulta integer NOT NULL,
  id_formas_de_pagamento integer not NULL,
  data_agenda date NOT NULL,
  Hora_agenda time NOT NULL,
  data_confirmacao date,
  hora_confirmacao time,
  observacoes varchar(255)
  --CONSTRAINT agenda_pkey PRIMARY KEY (id)
);

-- agenda foreign keys

ALTER TABLE agenda ADD CONSTRAINT agenda_id_cliente_fkey FOREIGN KEY
(id_cliente) REFERENCES clientes(id) ON DELETE RESTRICT;
ALTER TABLE agenda ADD CONSTRAINT agenda_id_convenio_fkey FOREIGN KEY
(id_convenio) REFERENCES convenios(id) ON DELETE RESTRICT;
ALTER TABLE agenda ADD CONSTRAINT agenda_id_formas_de_pagamento_fkey
FOREIGN KEY (id_formas_de_pagamento) REFERENCES formas_de_pagamento(id)
ON DELETE RESTRICT;
ALTER TABLE agenda ADD CONSTRAINT agenda_id_medico_fkey FOREIGN KEY
(id medico) REFERENCES medicos(id) ON DELETE RESTRICT;
```

```
ALTER TABLE agenda ADD CONSTRAINT  
agenda_id_status_da_agenda_ou_consulta_fkey FOREIGN KEY  
(id_status_da_agenda_ou_consulta) REFERENCES  
status_da_agenda_ou_consulta(id) ON DELETE RESTRICT;
```

-- consulta definition

```
CREATE TABLE IF NOT exists consulta (  
  Id serial primary key,  
  data date NOT NULL,  
  hora time NOT NULL,  
  id_agenda integer,  
  id_cliente integer NOT NULL,  
  id_medico integer NOT NULL,  
  id_convenio integer,  
  id_forma_de_pagamento integer not NULL,  
  id_status_da_agenda_ou_consulta integer NOT NULL,  
  observacao varchar(255)  
  --CONSTRAINT consulta_pkey PRIMARY KEY (id)  
);
```

-- consulta foreign keys

```
ALTER TABLE consulta ADD CONSTRAINT consulta_id_agenda_fkey FOREIGN KEY  
(id_agenda) REFERENCES agenda(id) ON DELETE RESTRICT;  
ALTER TABLE consulta ADD CONSTRAINT consulta_id_cliente_fkey FOREIGN KEY  
(id_cliente) REFERENCES clientes(id) ON DELETE RESTRICT;  
ALTER TABLE consulta ADD CONSTRAINT consulta_id_convenio_fkey FOREIGN KEY  
(id_convenio) REFERENCES convenios(id) ON DELETE RESTRICT;  
ALTER TABLE consulta ADD CONSTRAINT consulta_id_forma_de_pagamento_fkey  
FOREIGN KEY (id_forma_de_pagamento) REFERENCES formas_de_pagamento(id) ON  
DELETE RESTRICT;  
ALTER TABLE consulta ADD CONSTRAINT consulta_id_medico_fkey FOREIGN KEY  
(id_medico) REFERENCES medicos(id) ON DELETE RESTRICT;
```

-- servico_de_agendas definition

```
CREATE TABLE IF NOT exists servico_de_agendas (  
  Id serial primary key,  
  id_operador integer not NULL,  
  nome varchar(100) not NULL,  
  login varchar(50) not NULL,  
  senha varchar(50) not NULL  
  --CONSTRAINT servico_de_agendas_pkey PRIMARY KEY (id)  
);
```

-- servico_de_agendas foreign keys

```
ALTER TABLE servico_de_agendas ADD CONSTRAINT  
servico_de_agendas_id_operador_fkey FOREIGN KEY (id_operador) REFERENCES
```

```
operadores(id) ON DELETE RESTRICT;
```

```
-- hospitaais definition
```

```
CREATE TABLE IF NOT EXISTS hospitaais (
  Id serial primary key,
  nome varchar(50) not NULL,
  telefone varchar(20) not NULL
  --CONSTRAINT hospitaais_pkey PRIMARY KEY (id)
);
```

```
-- integracao definition
```

```
CREATE TABLE IF NOT EXISTS integracao (
  Id serial primary key,
  endereco_do_site varchar(255) NOT NULL,
  login varchar(50) NOT NULL,
  senha varchar(20) NOT NULL,
  status bool NOT NULL
  --CONSTRAINT integracao_pkey PRIMARY KEY (id)
);
```

```
-- "__dm_xtable__" Usada para testar os tipos de dados o ORM
```

- **Observações:**

- Essas chaves estrangeiras garantem a integridade referencial das tabelas, o que significa que os dados nas tabelas relacionadas serão consistentes.
- Por exemplo, a chave estrangeira fk_medico_operador na tabela medico garante que o valor da coluna id_operador em uma linha da tabela medico sempre existirá em uma linha da tabela operador.

Script SQL PostgreSQL para criar os índices das tabelas

```
-- "dm_xtable" Usada para testar os tipos de dados o ORM
```

```
COMMENT ON TABLE dm_xtable IS 'Tabela que demonstra o uso dos tipos de dados do projeto maricarai'; COMMENT ON COLUMN dm_xtable.id IS '\LLLLLL'; COMMENT ON COLUMN dm_xtable.Id_Operadores IS '\LLLLLL'; COMMENT ON COLUMN dm_xtable.nome IS '\ssssssssssssssssssssssssssssssssssssssssssssssssssssssssssss'; COMMENT ON COLUMN dm_xtable.endereco IS '\ssssssssssssssssssssssssssssssssssssssssssssssssssssssssss'; COMMENT ON COLUMN dm_xtable.cnpj IS '##.###.###/####-##'; COMMENT ON COLUMN dm_xtable.cpf IS '###.###.###-##'; COMMENT ON COLUMN dm_xtable.cep IS '##.###.###'; COMMENT ON COLUMN dm_xtable.valor_smallint IS '\IIIII'; COMMENT ON
```

```
COL CREATE TABLE IF NOT EXISTS dm_xtable (
id serial primary key, id_operadores integer,
nome varchar(50), endereco varchar(50), cnpj varchar(18), cpf varchar(14), cep varchar(10),
valor_smallint int2, valor_integer integer, valor_float8 float8, dd_mm_yy date, dd_mm_yyyy
date, dd_mm_yy_hh_nn timeStamp,
dd_mm_yy_hh_nn_ss timeStamp, dd_mm_yyyy_hh_nn timeStamp, dd_mm_yyyy_hh_nn_ss
timeStamp, hh_nn time, hh_nn_ss time,
sexo varchar(15), -- Indefinido; Masculino; Feminino estado_civil varchar(15), -- Amigado,
Solteiro; Casado; Divorciado; ativo varchar(5) -- False; True
```

```
--CONSTRAINT "__dm_xtable__pkey" PRIMARY KEY (id)
```

```
);
```

```
COMMENT ON TABLE dm_xtable IS 'Tabela que demonstra o uso dos tipos de dados do
projeto maricarai'; COMMENT ON COLUMN dm_xtable.id IS '\LLLLLL'; COMMENT ON
COLUMN dm_xtable.Id_Operadores IS '\LLLLLL'; COMMENT ON COLUMN dm_xtable.nome
IS '\ssssssssssssssssssssssssssssssssssssssssssssssssssssss'; COMMENT ON COLUMN
dm_xtable.endereco IS '\ssssssssssssssssssssssssssssssssssssssssssssssssssss'; COMMENT
ON COLUMN dm_xtable.cnpj IS '##.###.###/####-##'; COMMENT ON COLUMN
dm_xtable.cpf IS '###.###.###-##'; COMMENT ON COLUMN dm_xtable.cep IS
'##.###.###'; COMMENT ON COLUMN dm_xtable.valor_smallint IS '\IIIII'; COMMENT ON
COLUMN dm_xtable.valor_integer IS '\LLLLLLLL'; COMMENT ON COLUMN
dm_xtable.valor_float8 IS '\RRR,RRR,RRR.RR'; COMMENT ON COLUMN
dm_xtable.dd_mm_yy IS 'Ddd/mm/yy'; COMMENT ON COLUMN dm_xtable.dd_mm_yyyy IS
'Ddd/mm/yyyy'; COMMENT ON COLUMN dm_xtable.dd_mm_yy_hh_nn IS 'Ddd/mm/yy
hh:nn'; COMMENT ON COLUMN dm_xtable.dd_mm_yy_hh_nn_ss IS 'Ddd/mm/yy hh:nn:ss';
COMMENT ON COLUMN dm_xtable.dd_mm_yyyy_hh_nn IS 'Ddd/mm/yyyy hh:nn';
COMMENT ON COLUMN dm_xtable.dd_mm_yyyy_hh_nn_ss IS 'Ddd/mm/yyyy hh:nn:ss';
COMMENT ON COLUMN dm_xtable.hh_nn IS 'Dhh:nn'; COMMENT ON COLUMN
dm_xtable.hh_nn_ss IS 'Dhh:nn:ss'; COMMENT ON COLUMN dm_xtable.sexo IS
'\ssssssssssssss'; COMMENT ON COLUMN dm_xtable.estado_civil IS '\ssssssssssssss';
COMMENT ON COLUMN dm_xtable.ativo IS '\sssss'; UMN dm_xtable.valor_integer IS
'\LLLLLLLL'; COMMENT ON COLUMN dm_xtable.valor_float8 IS '\RRR,RRR,RRR.RR';
COMMENT ON COLUMN dm_xtable.dd_mm_yy IS 'Ddd/mm/yy'; COMMENT ON COLUMN
dm_xtable.dd_mm_yyyy IS 'Ddd/mm/yyyy'; COMMENT ON COLUMN
dm_xtable.dd_mm_yy_hh_nn IS 'Ddd/mm/yy hh:nn'; COMMENT ON COLUMN
dm_xtable.dd_mm_yy_hh_nn_ss IS 'Ddd/mm/yy hh:nn:ss'; COMMENT ON COLUMN
dm_xtable.dd_mm_yyyy_hh_nn IS 'Ddd/mm/yyyy hh:nn'; COMMENT ON COLUMN
dm_xtable.dd_mm_yyyy_hh_nn_ss IS 'Ddd/mm/yyyy hh:nn:ss'; COMMENT ON COLUMN
dm_xtable.hh_nn IS 'Dhh:nn'; COMMENT ON COLUMN dm_xtable.hh_nn_ss IS 'Dhh:nn:ss';
COMMENT ON COLUMN dm_xtable.sexo IS '\ssssssssssssss'; COMMENT ON COLUMN
dm_xtable.estado_civil IS '\ssssssssssssss'; COMMENT ON COLUMN dm_xtable.ativo IS
'\sssss'; UMN dm_xtable.dd_mm_yyyy IS 'Ddd/mm/yyyy'; COMMENT ON COLUMN
dm_xtable.dd_mm_yy_hh_nn IS 'Ddd/mm/yy hh:nn'; COMMENT ON COLUMN
```

```
dm_xtable.dd_mm_yy_hh_nn_ss IS 'Ddd/mm/yy hh:nn:ss'; COMMENT ON COLUMN
dm_xtable.dd_mm_yyyy_hh_nn IS 'Ddd/mm/yyyy hh:nn'; COMMENT ON COLUMN
dm_xtable.dd_mm_yyyy_hh_nn_ss IS 'Ddd/mm/yyyy hh:nn:ss'; COMMENT ON COLUMN
dm_xtable.hh_nn IS 'Dhh:nn'; COMMENT ON COLUMN dm_xtable.hh_nn_ss IS 'Dhh:nn:ss';
COMMENT ON COLUMN dm_xtable.sexo IS '\ssssssssssssss'; COMMENT ON COLUMN
dm_xtable.estado_civil IS '\ssssssssssssss'; COMMENT ON COLUMN dm_xtable.ativo IS
'\sssss'; UMN dm_xtable.valor_float8 IS '\RRR,RRR,RRR.RR'; COMMENT ON COLUMN
dm_xtable.dd_mm_yy IS 'Ddd/mm/yy'; COMMENT ON COLUMN dm_xtable.dd_mm_yyyy IS
'Ddd/mm/yyyy'; COMMENT ON COLUMN dm_xtable.dd_mm_yy_hh_nn IS 'Ddd/mm/yy
hh:nn'; COMMENT ON COLUMN dm_xtable.dd_mm_yy_hh_nn_ss IS 'Ddd/mm/yy hh:nn:ss';
COMMENT ON COLUMN dm_xtable.dd_mm_yyyy_hh_nn IS 'Ddd/mm/yyyy hh:nn';
COMMENT ON COLUMN dm_xtable.dd_mm_yyyy_hh_nn_ss IS 'Ddd/mm/yyyy hh:nn:ss';
COMMENT ON COLUMN dm_xtable.hh_nn IS 'Dhh:nn'; COMMENT ON COLUMN
dm_xtable.hh_nn_ss IS 'Dhh:nn:ss'; COMMENT ON COLUMN dm_xtable.sexo IS
'\ssssssssssssss'; COMMENT ON COLUMN dm_xtable.estado_civil IS '\ssssssssssssss';
COMMENT ON COLUMN dm_xtable.ativo IS '\sssss';
```

- ****Observações****:

- Essas chaves estrangeiras garantem a integridade referencial das tabelas, o que significa que os dados nas tabelas relacionadas serão consistentes.
- Por exemplo, a chave estrangeira `fk_medico_operador` na tabela `medico` garante que o valor da coluna `id_operador` em uma linha da tabela `medico` sempre existirá em uma linha da tabela `operador`.

Script SQL PostgreSQL para criar os índices das tabelas

```SQL

- **Observações:**

- **Observações:**

- Após criar as tabelas e as chaves estrangeiras, você poderá inserir dados nas tabelas. Os dados que você insere devem corresponder aos atributos das classes no diagrama de classes.
- Por exemplo, para inserir um médico na tabela `medico`, você precisará fornecer os valores para os seguintes atributos:
  - `id` (auto-incrementado)
  - `id_operador`
  - `nome`
  - `telefone`
  - `telefone_secretaria`
  - `login`

- senha
- Exemplos de consultas que você pode usar para trabalhar com os dados no banco de dados:
  - Para obter uma lista de todos os médicos:

```
SELECT *
FROM medico;
```

- Para obter uma lista de todos os médicos do hospital 1:

```
SELECT *
FROM medico
WHERE id_hospital = 1;
```

- Para obter uma lista de todas as consultas do paciente 1:

```
SELECT *
FROM consulta
WHERE id_cliente = 1;
```

- Para obter uma lista de todas as consultas do médico 1:

```
SELECT *
FROM consulta
WHERE id_medico = 1;
```

## Compartilhar o acesso ao banco de dados

---

- Conceder acesso ao banco de dados:

```
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO mypartner;
```

- Este comando concederá ao seu parceiro todos os privilégios no banco de dados mydatabase.

## Ferramenta que pode ser usada para executar os scripts acima?

---

- *psql*

- Para executar os scripts acima usando o *psql*, você precisará conectar-se ao banco de dados PostgreSQL. Você pode fazer isso usando o comando *psql*:

```
psql -h <host> -p <port> -d <database> -U <username>
```

- Por exemplo, para conectar-se ao banco de dados PostgreSQL chamado *mydatabase* no host *localhost* na porta *5432*, você usaria o seguinte comando:

```
psql -h localhost -p 5432 -d mydatabase -U myuser
```

- Uma vez conectado ao banco de dados, você pode executar os scripts acima usando o comando *\i*:

```
\i create_tables.sql
```

- Nota:

- Este comando irá executar o script *create\_tables.sql*, que criará as tabelas necessárias para o diagrama de classes.

- *dbeaver-ce*

- O *DBeaver Community Edition* (DBeaver CE) é uma ferramenta de gerenciamento de banco de dados relacional de código aberto. Ele oferece suporte a vários bancos de dados, como MySQL, PostgreSQL, SQLite, Oracle, Microsoft SQL Server, e muitos outros. O DBeaver CE é uma versão gratuita e de código aberto da ferramenta, projetada para ser usada por desenvolvedores, administradores de banco de dados e qualquer pessoa envolvida no gerenciamento e manipulação de bancos de dados.

- Principais características do DBeaver CE:

- Conectividade com Múltiplos Bancos de Dados: Oferece suporte a uma ampla variedade de sistemas de gerenciamento de banco de dados, permitindo que os usuários se conectem a diferentes bancos de dados a partir de uma única interface.
- Interface Gráfica Intuitiva: Possui uma interface de usuário gráfica intuitiva que facilita o desenvolvimento, gerenciamento e consulta de

bancos de dados.

- **Editor SQL:** Inclui um editor SQL poderoso com realce de sintaxe, conclusão automática e outras funcionalidades que facilitam a escrita e execução de consultas SQL.
  - **Gerenciamento de Metadados:** Permite a visualização e edição de metadados do banco de dados, como tabelas, índices, procedimentos armazenados, entre outros.
  - **Exportação e Importação de Dados:** Facilita a transferência de dados entre diferentes fontes, com suporte a várias opções de exportação e importação.
  - **Visualização de Dados:** Oferece recursos visuais para a visualização de dados, incluindo gráficos e ferramentas de exploração de dados.
  - **Suporte a Plugins:** Permite a extensão de funcionalidades por meio de plugins, proporcionando flexibilidade para atender a diferentes necessidades.
- O DBeaver CE é uma escolha popular devido à sua versatilidade e ao fato de ser gratuito e de código aberto. No entanto, é importante observar que existem versões comerciais do DBeaver (DBeaver EE) que oferecem recursos adicionais e suporte aprimorado, mas a versão Community Edition é uma opção robusta para muitos desenvolvedores e administradores de banco de dados.

## Referências

---

1. [Gráfico criado com projeto mermaid](#)
2. [PostgreSQL: The World's Most Advanced Open Source](#)

[ ](#topo "Retorna ao topo")